

UnBemEstar

Documento de Arquitetura

Versão 0.9

Tabela - Integrantes do Grupo

Matrícula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
241025720	Vitor Eduardo Araújo	Líder, Documentação e Scrum Master	10
242028815	Breno Elias de Carvalho Correia	Arquitetura e DevOps	10
241038791	Flavia de Melo Rebelato	Product Owner	10
241012098	Brenda Maria Cavalcante Chaves	Desenvolvedor Front-End	10
251016045	Thiago da Silva Borges	Desenvolvedor Back-End	10
241025784	João Paulo da Silva Pereira	Documentação e Requisitos	10
241025819	Pablo Antonio Martins de Sousa	Arquitetura e DevOps	10
241041204	PEDRO VICTOR TEIXEIRA SILVA	Desenvolvedor Back-End	10
231011023	Alexandre Vilar Valadares Fonsêca	Desenvolvedor Front-End	10
222006866	João Vitor Tavares de Sá Lima	Arquitetura e DevOps	10

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
06/05/2026	0.5	Primeira reunião do grupo, finalizando o documento de visão e iniciando o documento de arquitetura	Grupo completo
13/05/2026	0.7	Revisão do documento e ajustes finais para a última versão.	Thiago, Pablo Antônio
15/05/2026	0.9	Revisão final e Formatação.	Vitor Araújo, Brenda Chave

Sumário

1	Introdução.....	3
1.1	Propósito	4
1.2	Escopo	4
2	Representação Arquitetural.....	4
2.1	Definições	4
2.2	Justificativa da Escolha.	4
2.3	Metas e restrições arquiteturais	9
	As metas e restrições a seguir orientam as decisões arquiteturais e o desenvolvimento do UnBemEstar, derivando dos requisitos não funcionais definidos no Documento de Visão (seção 4.4) e de boas práticas adotadas pela equipe.	9
2.4	Visões.....	9
2.4.1	Visão de uso	10
2.4.2	Visão de organização lógica	12
2.4.3	Visão Estrutural	13
	A composição do UnBemEstar e as relações entre suas partes baseiam-se em componentes distribuídos para atender aos requisitos operacionais e escalar com facilidade:.....	13
2.5	Visão de Implantação	16
2.6	Restrições adicionais	18
2.6.1	Restrições Negociais e de Contexto de Uso	18
2.6.2	Restrições de Qualidade de Software	18
2.6.3	Restrições de Segurança e Permissões de Acesso	19
3	Bibliografia.....	20

1 Introdução

1.1 Propósito

O presente documento tem como propósito descrever a arquitetura de software do sistema UnBemEstar, desenvolvido como projeto acadêmico da disciplina Metodos de Desenvolvimento de Software 2026.1, no curso de Engenharia de Software da UNB. Este documento tem como função central registrar as decisões arquiteturais tomadas durante o planejamento e a implementação do sistema, incluindo a definição do estilo arquitetural adotado, a organização dos módulos, as tecnologias envolvidas e as restrições técnicas aplicáveis. O conteúdo aqui apresentado servirá como guia para a equipe de desenvolvimento, facilitando a comunicação entre os membros do grupo, manutenção do código e evolução futura do produto.

Adicionalmente, este documento busca assegurar que as decisões técnicas estejam alinhadas com os objetivos funcionais e de qualidade do projeto, especialmente no que se refere à proteção dos dados pessoais e clínicos dos usuários em conformidade com a LGPD, à usabilidade da interface para públicos de diferentes faixas etárias e níveis de familiaridade com tecnologia, e à automação dos fluxos de atendimento em clínicas de fisioterapia

1.2 Escopo

A UnBemEstar é uma plataforma web de gestão e automação de agendamento voltada para clínicas de fisioterapia, cujo objetivo é centralizar o controle operacional e eliminar a dependência de processos manuais conduzidos por rede sociais e aplicativos de mensagem. O sistema contempla quatro perfis de usuário: paciente, secretária, fisioterapeuta/médico e administrador, e oferece funcionalidades como autenticação, calendário interativo, agendamento autônomo de consultas presenciais e domiciliares, lembretes automáticos por e-mail, upload de documentos clínicos e registro de evolução por sessão. O detalhamento completo dos requisitos funcionais e não funcionais, dos perfis de acesso e dos cenários de uso encontra-se no Documento de Visão de Produto e Projeto, seção 4.

2 Representação Arquitetural

Este capítulo descreve a arquitetura de software do UnBemEstar, detalhando o estilo arquitetural adotado, seus componentes e as decisões que guiaram sua escolha.

2.1 Definições

O sistema UnBemEstar seguirá uma arquitetura em camadas, implantada sob a forma de monólito modular com execução serverless. A arquitetura é composta pelas camadas de apresentação, aplicação (API), serviços de negócio, acesso a dados e persistência, integradas a serviços externos especializados para envio de e-mails transacionais (Resend) e execução de tarefas agendadas (Vercel Cron Jobs).

2.2 Justificativa da Escolha.

A escolha arquitetural fundamenta-se nas características do produto e do projeto descritas no documento de visão. A arquitetura em camadas atende à necessidade de separação clara de responsabilidades entre apresentação, regras de negócio e persistência, característica fundamental dada a coexistência de quatro perfis de usuário com permissões distintas, e favorece a manutenibilidade do código e a aplicação das práticas de Extreme Programming adotadas pela equipe (seção 3 do Documento de Visão). A opção por um monólito modular, em detrimento de microsserviços, vem do escopo controlado do produto, do prazo semestral e da composição da equipe (dez integrantes). Uma arquitetura distribuída introduziria complexidade operacional incompatível com o contexto e elevaria os riscos mapeados na seção 2.5 do Documento de Visão. A execução Serverless, viabilizada pelo deploy na Vercel, adere nativamente ao framework Next.js e atende aos requisitos de pipeline de entrega contínua (RNF05). A natureza das funções serverless contribui para o cumprimento do requisito de desempenho (RNF04). A integração com o Supabase, que concentra banco de dados, autenticação e armazenamento em uma única plataforma gerenciada, reduz a quantidade de componentes de infraestrutura sob responsabilidade da equipe

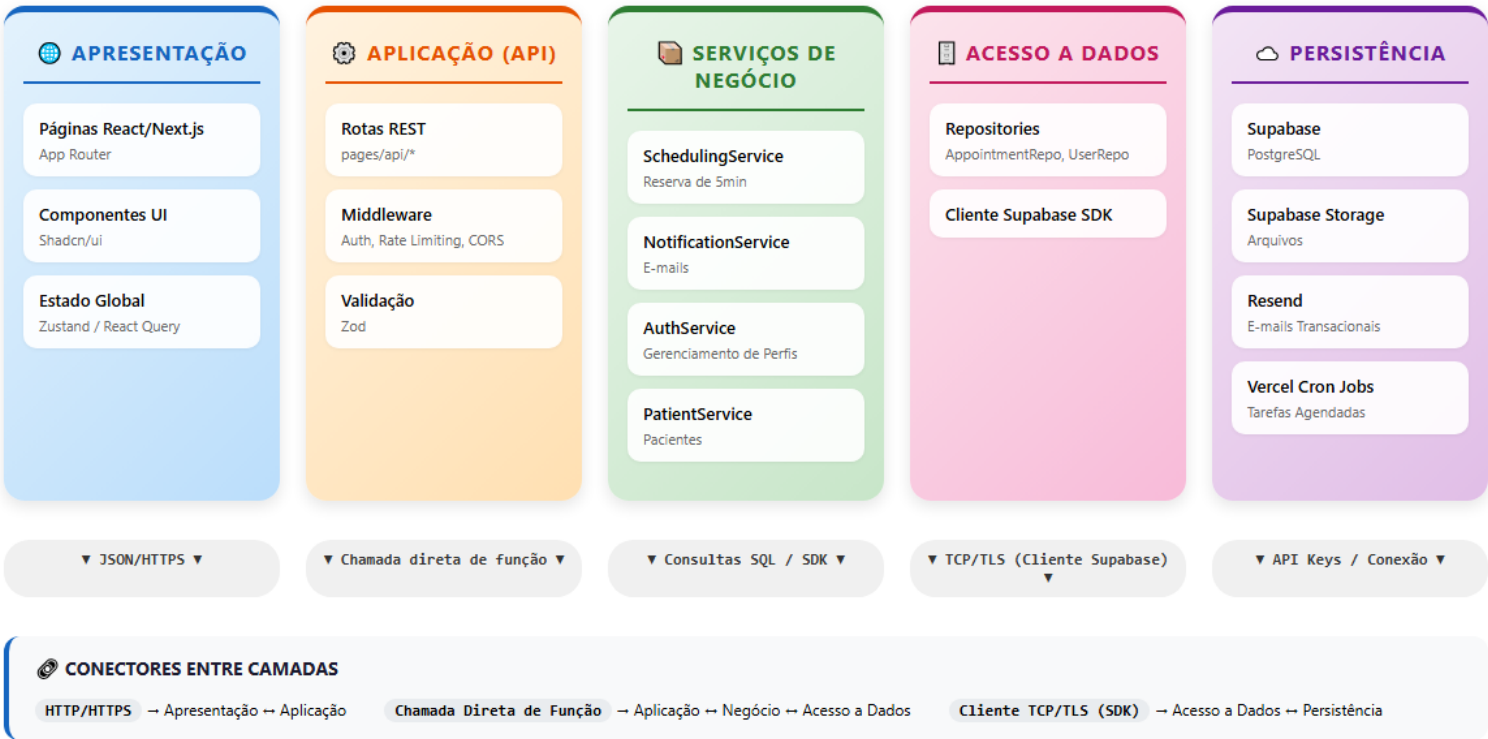
2.3 Detalhamento

A escolha pela arquitetura em camadas com monólito modular serverless é detalhada a seguir por meio de uma figura esquemática, que evidencia os componentes, interfaces e conectores do sistema UnBemEstar.

2.3.1 Figura Esquemática da Arquitetura

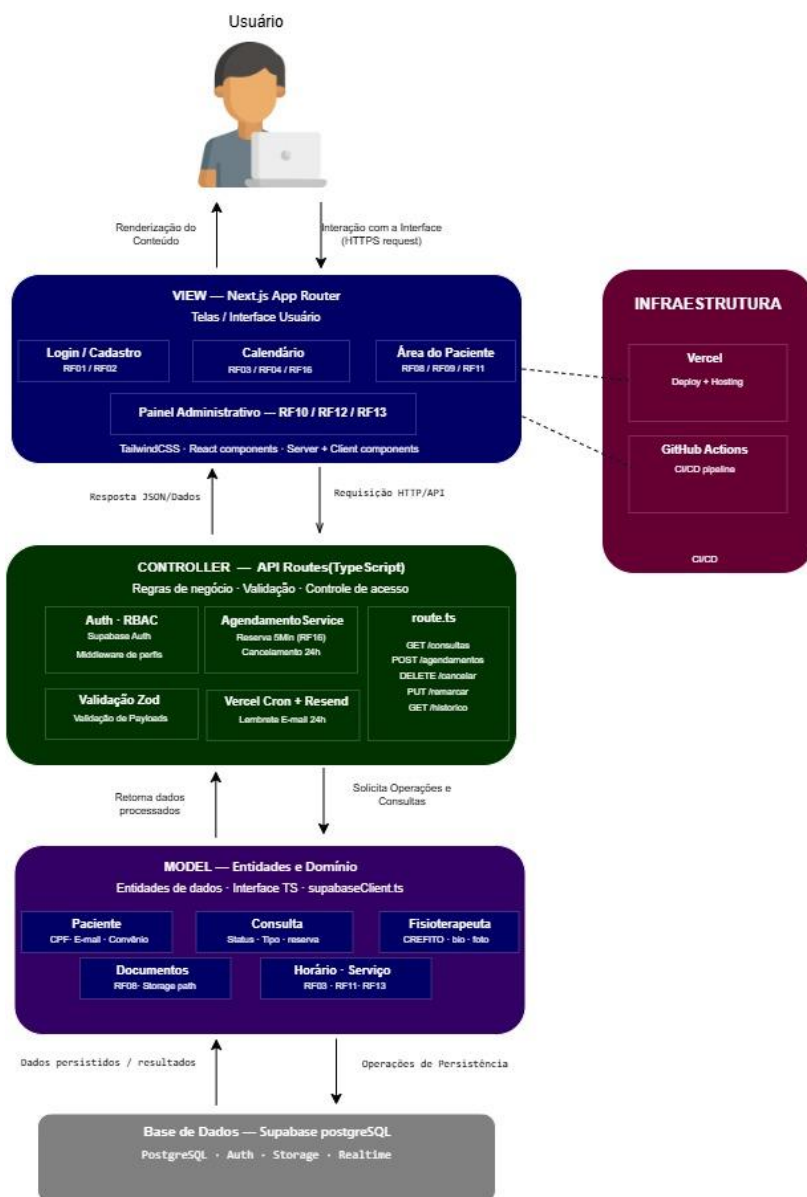
A Figura 1 detalha o estilo arquitetural em camadas adotado, evidenciando a separação de responsabilidades, os componentes principais, suas interfaces e os conectores que viabilizam a comunicação.

Figura 1 - Esquema da Arquitetura em Camadas do UnBemEstar



Descrição da Figura 1: A arquitetura é organizada em cinco camadas verticais: Apresentação, Aplicação (API), Serviços de Negócio, Acesso a Dados e Persistência. As setas indicam o fluxo de requisição (descendente) e resposta (ascendente). A camada de Persistência se conecta a serviços externos gerenciados (Supabase, Resend, Vercel Cron Jobs). Os conectores principais são: HTTP/HTTPS (entre Apresentação e Aplicação), Chamada Direta de Função (entre Aplicação, Negócio e Acesso a Dados) e Cliente TCP/TLS (entre Acesso a Dados e Persistência).

Figura 2 - Diagrama Arquitetural MVC com Detalhamento Tecnológico



Fonte: Elaborado por (2026).

Complementando a visão em camadas apresentada na Figura 1, a Figura 2 detalha a arquitetura do UnBemEstar sob a perspectiva do padrão MVC (Model-View-Controller), evidenciando os componentes tecnológicos específicos adotados em cada camada.

A figura organiza a arquitetura em quatro blocos principais:

- **View (Next.js App Router):** Responsável pela interface com o usuário, incluindo telas de login, calendário, área do paciente e painel administrativo. Utiliza TailwindCSS, componentes React Server e Client.
- **Controller (API Routes TypeScript):** Gerencia as regras de negócio, validação (Zod) e controle de acesso (RBAC com Supabase Auth). Inclui os serviços de agendamento (reserva de 5 minutos, cancelamento) e integração com Vercel Cron + Resend para lembretes.
- **Model (Entidades e Domínio):** Define as entidades de dados (Paciente, Consulta, Fisioterapeuta, Documentos, Horário) com suas interfaces TypeScript e cliente Supabase.
- **Infraestrutura:** Camada base que sustenta o sistema, incluindo Vercel (deploy/hosting), GitHub Actions (CI/CD) e Supabase (banco de dados, storage e autenticação).

2.3.2 Instanciação dos Elementos Arquiteturais

Aplicando o estilo de camadas ao contexto do UnBemEstar, tem-se a seguinte instanciação concreta:

- **Camada de Apresentação (Front-end):** Composta por páginas (app/dashboard/, app/schedule/), componentes React (Calendar.tsx, PatientForm.tsx) e hooks customizados (useAuth, useAppointments). Suas interfaces são as props de componentes React e chamadas assíncronas para os hooks que consomem a API. Os conectores utilizam Fetch API ou Axios (HTTP/HTTPS) para consumir as rotas da API Next.js.
- **Camada de Aplicação (API Routes):** Materializada nos arquivos de rota em pages/api/auth/[...nextauth].ts, pages/api/appointments/book.ts e pages/api/patients/upload-document.ts. As interfaces são requisições HTTP (método, headers, body validado via Zod) e respostas JSON padronizadas (ex: { success: true, data: ... } ou { error: ... }). Os conectores realizam chamadas diretas de funções TypeScript para os Services na camada de negócio.
- **Camada de Serviços de Negócio:** Implementada por classes ou módulos como SchedulingService (lógica de reserva de 5 min), NotificationService (integração com Resend) e AuthService (gerenciamento de perfis). As interfaces são métodos públicos como bookAppointment(data: BookingDTO): Promise<Appointment> ou sendReminder(appointmentId: string): Promise<void>. Os conectores realizam chamadas para os Repositories na camada de acesso a dados.
- **Camada de Acesso a Dados:** Composta pelos repositórios AppointmentRepository e pelo cliente Singleton do Supabase (getSupabaseClient). As interfaces são métodos como create(data): Promise<Appointment> e findOverlapping(params):

Promise<boolean>, utilizando tipos gerados a partir do esquema do banco. Os conectores utilizam o SDK do Supabase (que gerencia a conexão TCP/TLS com o PostgreSQL e Storage).

- **Camada de Persistência (Serviços Externos):** Constituída pelo Banco de Dados Supabase (PostgreSQL), Storage Supabase (para documentos e fotos), API do Resend e Vercel Cron Jobs. As interfaces são conexões via string de conexão (banco) ou chaves de API (Resend, Storage, Cron), seguindo contratos RESTful ou específicos do provedor.

2.3.3 Responsabilidades, Justificativas e Regras de Uso

- **Camada de Apresentação:** Existe para isolar a lógica de interface do usuário (UI/UX) do restante do sistema, permitindo evoluir a aparência sem afetar as regras de negócio, justificando-se pelos requisitos de usabilidade para diferentes faixas etárias (RNF02). Seu papel é renderizar a UI, capturar entradas do usuário, exibir dados e feedback, e traduzir ações do usuário em chamadas de API. Como regras de uso, deve se comunicar apenas com a Camada de Aplicação via HTTP, não podendo conter lógica de negócio. Sua entrada esperada são cliques e dados inseridos pelo usuário, e sua saída produzida são requisições HTTP para a API.
- **Camada de Aplicação (API Routes):** Atua como um ponto de entrada único e seguro para o front-end, centralizando a validação inicial (Zod), autenticação (JWT) e autorização (perfis), alinhando-se à meta de segurança e LGPD (RNF01). Seu papel é receber requisições, autenticar o usuário, validar os dados de entrada, orquestrar a chamada aos Services de negócio apropriados e formatar a resposta HTTP. Cada rota deve validar o body/query com um schema Zod e verificar a sessão do usuário via Supabase Auth. A entrada esperada é uma requisição HTTP com headers (Authorization: Bearer) e body JSON; a saída é uma resposta HTTP JSON padronizada. Para funcionar, requer as chaves de ambiente do Supabase e do Resend.
- **Camada de Serviços de Negócio:** É o coração do sistema, encapsulando as regras de negócio mais importantes, como a lógica de "reserva temporária de 5 minutos" (RF16) e o cálculo de ocupação da agenda. Isolar essa lógica facilita testes e manutenção. Seu papel é implementar a lógica específica do domínio de fisioterapia/agendamento e orquestrar operações complexas que envolvem múltiplos repositórios. Não deve conhecer detalhes de HTTP ou da UI. A entrada esperada são dados de domínio validados semanticamente; a saída são resultados de operações (entidades, booleanos, listas) ou erros de regra de negócio.
- **Camada de Acesso a Dados (Repositórios):** Abstrai a complexidade e a tecnologia específica de persistência (Supabase/PostgreSQL), permitindo que o serviço de negócio não precise saber SQL ou os detalhes do cliente Supabase. Seu papel é executar operações CRUD e consultas específicas no banco e no storage. Cada repositório é responsável por uma entidade principal e deve usar o cliente Supabase injetado. A entrada esperada são filtros, IDs ou objetos parciais; a saída são entidades completas ou listas. Para funcionar, requer as variáveis de ambiente SUPABASE_URL e SUPABASE_ANON_KEY.
- **Camada de Persistência (Serviços Externos):** Delega a responsabilidade de armazenamento durável, gerenciamento de e-mails e tarefas agendadas a plataformas

especializadas e gerenciadas, reduzindo a carga operacional da equipe. Seu papel é fornecer armazenamento ACID para dados do sistema (PostgreSQL), armazenamento de arquivos (Storage), envio confiável de e-mails transacionais (Resend) e execução de rotinas temporizadas (Cron Jobs). São acessados exclusivamente pela Camada de Acesso a Dados ou pelos Cron Jobs. Para funcionar, necessitam de credenciais de serviço (Service Role Key, API Keys) e conectividade de rede externa.

2.3 Metas e restrições arquiteturais

As metas e restrições a seguir orientam as decisões arquiteturais e o desenvolvimento do UnBemEstar, derivando dos requisitos não funcionais definidos no Documento de Visão (seção 4.4) e de boas práticas adotadas pela equipe.

Metas de qualidade

- **Desempenho:** tempo de resposta inferior a 3 segundos nas operações principais (RNF04).
- **Disponibilidade:** acesso pelos principais navegadores web em versões recentes (RNF03).
- **Segurança e LGPD:** autenticação com hash criptográfico, controle de acesso por Row Level Security (RLS) no Supabase e tráfego exclusivamente por HTTPS (RNF01).
- **Cobertura de testes:** mínimo de 70% por testes automatizados (seção 5.3.5 do Documento de Visão).

Restrições tecnológicas

- Front-end e back-end em **TypeScript** sobre **Next.js**.
- Persistência exclusivamente via **Supabase (PostgreSQL gerenciado)**.
- Envio de e-mails via **Resend** e tarefas agendadas via **Vercel Cron Jobs**.
- Deploy exclusivamente na **Vercel**, com pipeline CI/CD acoplada ao GitHub (RNF05).

Padrões de código e API

- TypeScript em modo estrito; ESLint e Prettier com execução obrigatória em pré-commit e CI.
- Validação de entrada em todas as rotas via **Zod**.
- Rotas REST com uso semântico de verbos HTTP e respostas JSON padronizadas.
- Endpoints sensíveis exigem autenticação via Supabase Auth; rotas críticas protegidas por *rate limiting*.

Versionamento

- Política **GitFlow simplificado** (main, develop, feature/*, hotfix/*).
- Mensagens de commit no padrão **Conventional Commits**.
- Integração somente via Pull Request com revisão de pelo menos um par e CI aprovada.

2.4 Visões

Esta seção apresenta o sistema UnBemEstar sob diferentes perspectivas arquiteturas, cada uma destinada a um público específico e a um conjunto particular de preocupações. As visões estão organizadas conforme o padrão IEEE 1471-2000, que define uma visão como uma representação do sistema a partir da perspectiva de um conjunto relacionado de preocupações.

2.4.1 Visão de uso

O sistema UnBemEstar consiste em uma plataforma web dedicada à automação e gestão de clínicas de fisioterapia. Seu escopo resume-se em centralizar a jornada do paciente desde o agendamento autônomo com reserva temporária de horários até o acompanhamento de prontuários e evolução clínica enquanto oferece aos gestores ferramentas para maximizar a ocupação da agenda e reduzir a ociosidade causada por desistências.

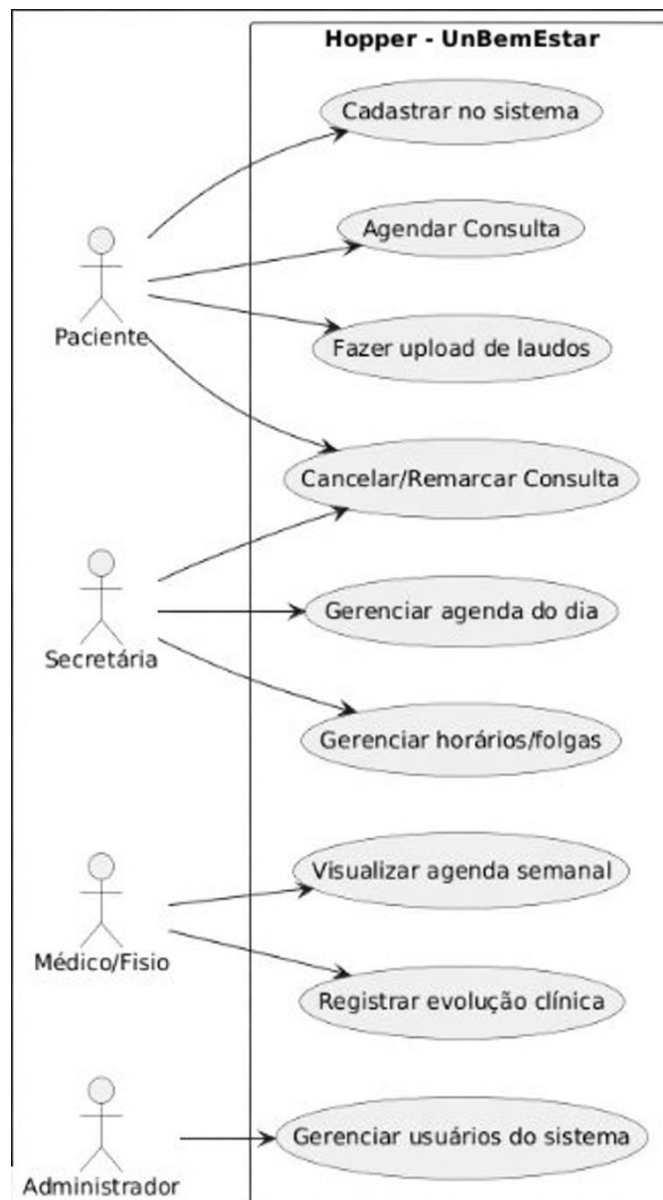
A escolha pelo estilo arquitetural **Cliente-Servidor** com foco em **BaaS** (Backend as a Service) foi guiada por requisitos críticos identificados na visão do produto:

- **Desempenho (RNF04):** A necessidade de resposta em menos de 3 segundos exige uma stack eficiente como Next.js.
- **Segurança e LGPD (RNF01):** O tratamento de dados sensíveis de saúde demanda uma infraestrutura de persistência robusta e segura, como o Supabase.
- **Agilidade e Entrega Contínua:** A estrutura modular facilita as práticas de *Extreme Programming* (XP) adotadas pela equipe, permitindo deploys frequentes via Vercel.

Justificativa adicional baseada na experiência da equipe: A escolha das tecnologias também se justifica pelo conhecimento prévio dos membros da equipe. Parte dos desenvolvedores já possuía experiência com React.js e TypeScript em projetos anteriores, o que acelera o ciclo de desenvolvimento e reduz a curva de aprendizado. Adicionalmente, dois integrantes já haviam utilizado o Supabase em projetos acadêmicos, trazendo familiaridade com suas funcionalidades de autenticação, Row Level Security (RLS) e integração com Next.js. A Vercel, por sua vez, foi selecionada por sua simplicidade de deploy e integração nativa com o framework, além de já ser conhecida por parte da equipe. Esse conhecimento prévio permitiu à equipe focar esforços nas regras de negócio e nas funcionalidades centrais do sistema, em vez de perder tempo com configurações complexas de infraestrutura.

Figura 3 - Diagrama de Casos de Uso

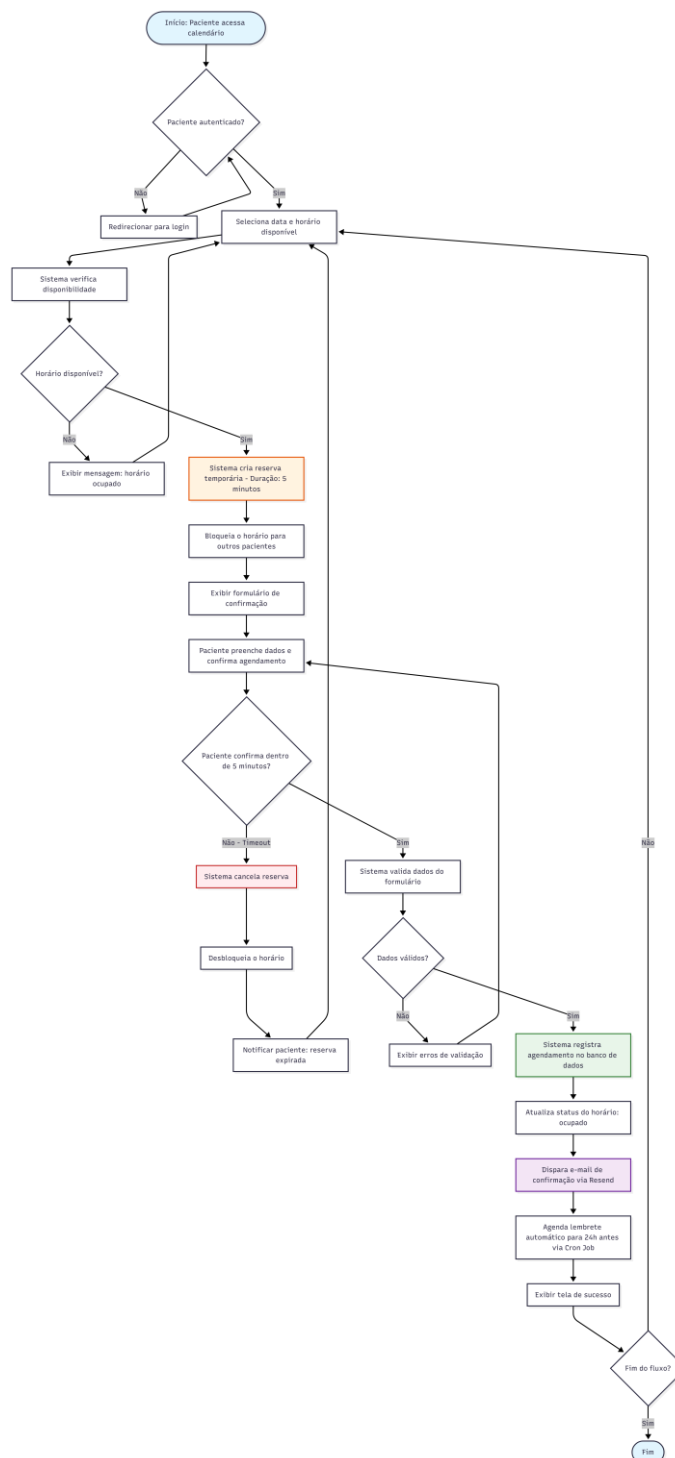
O diagrama apresenta os atores do sistema (Paciente, Secretária, Fisioterapeuta, Administrador) e os principais casos de uso associados a cada perfil, incluindo agendamento, cancelamento, registro de evolução e geração de relatórios.



Fonte: Elaborado por Breno Elias (2026)

Figura 4 – Diagrama de Atividades

O diagrama ilustra o fluxo da funcionalidade mais complexa do sistema: o agendamento de consulta com reserva temporária de 5 minutos (RF16). O fluxo contempla autenticação, seleção de horário, criação de reserva, confirmação com validação de dados, persistência no Supabase, envio de e-mail via Resend e agendamento de lembrete via Vercel Cron Job, além do tratamento de timeout e cancelamento da reserva.



Fonte: Elaborado por Brenda Maria Cavalcante Chaves (2026).

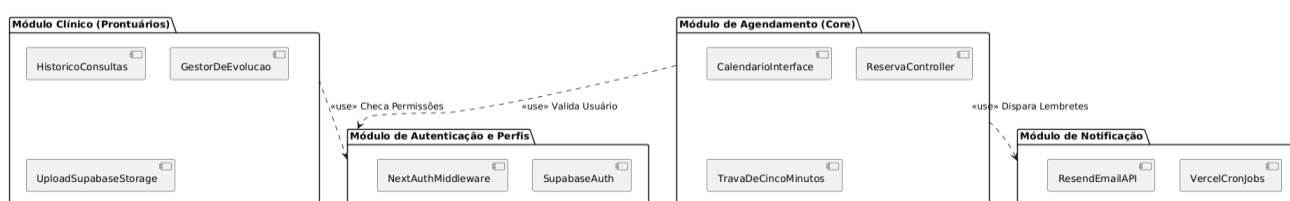
2.4.2 Visão de organização lógica

O software é organizado em módulos independentes que se comunicam para garantir a fluidez do sistema. A comunicação entre o front-end e o back-end ocorre via requisições **HTTP RESTful**, utilizando **JSON** como formato de troca de dados e o cliente nativo do Supabase para operações de banco.

A subdivisão lógica compreende:

1. **Módulo de Autenticação e Perfis:** Controla o acesso seguro de Pacientes, Secretárias, Médicos e Administradores.
Razão Lógica: Garantir a privacidade dos dados e que cada ator acesse apenas suas funcionalidades permitidas.
2. **Módulo de Agendamento (Core):** Gerencia o calendário interativo e a regra de reserva temporária de 5 minutos (RF16).
Razão Lógica: Evitar o choque de horários (overbooking) e automatizar a ocupação de lacunas na agenda.
3. **Módulo Clínico (Prontuário e Evolução):** Lida com o registro de sessões (RF10) e upload de documentos (RF08).
Razão Lógica: Centralizar as informações de saúde do paciente para facilitar o acompanhamento terapêutico.
4. **Módulo de Notificação:** Integra-se ao serviço Resend para envio de lembretes automáticos (RF07).
Razão Lógica: Reduzir as taxas de absenteísmo através de avisos prévios de 24 horas.

Figura 5 – Diagrama de Pacotes



Fonte: Elaborado por Breno Elias (2026)

2.4.3 Visão Estrutural

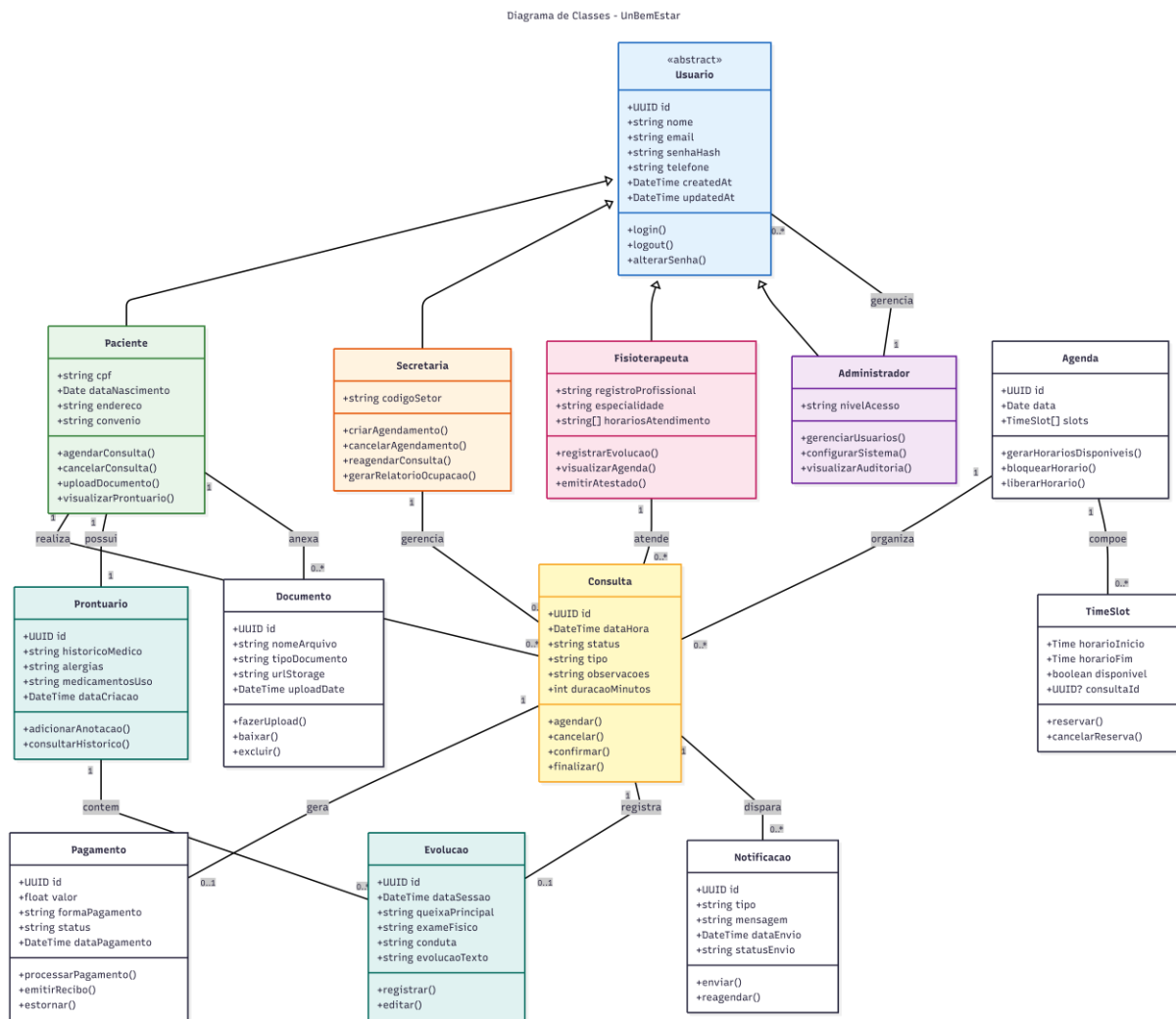
A composição do UnBemEstar e as relações entre suas partes baseiam-se em componentes distribuídos para atender aos requisitos operacionais e escalar com facilidade:

- **Client (Navegador Web):** Onde a interface construída em React.js renderiza a aplicação. Sua responsabilidade é lidar com as interações visuais, apresentar dados de forma responsiva e fazer as requisições HTTP para a camada de aplicação.
- **Controlador/API (Next.js API Routes):** Intermediário construído em TypeScript. Recebe requisições, valida dados de formulários utilizando a biblioteca Zod e aplica as regras de negócio antes de conversar com o banco de dados.
- **Persistência (Supabase):** Serviço que atua como o Banco de Dados relacional PostgreSQL gerenciado. Tem a responsabilidade de guardar com segurança todas as tabelas (pacientes, horários, configurações). A conexão é estabelecida utilizando o cliente nativo do Supabase no ambiente das rotas de API.

- **Interfaces de Terceiros:** Ferramentas adicionais conectadas ao *back-end*, com responsabilidades específicas: *Resend* (exclusivo para gerenciar o envio das mensagens de e-mail de confirmação e alerta) e *Vercel Cron* (para engatilhar rotinas baseadas em tempo).

Figura 6 - Diagrama de Classes

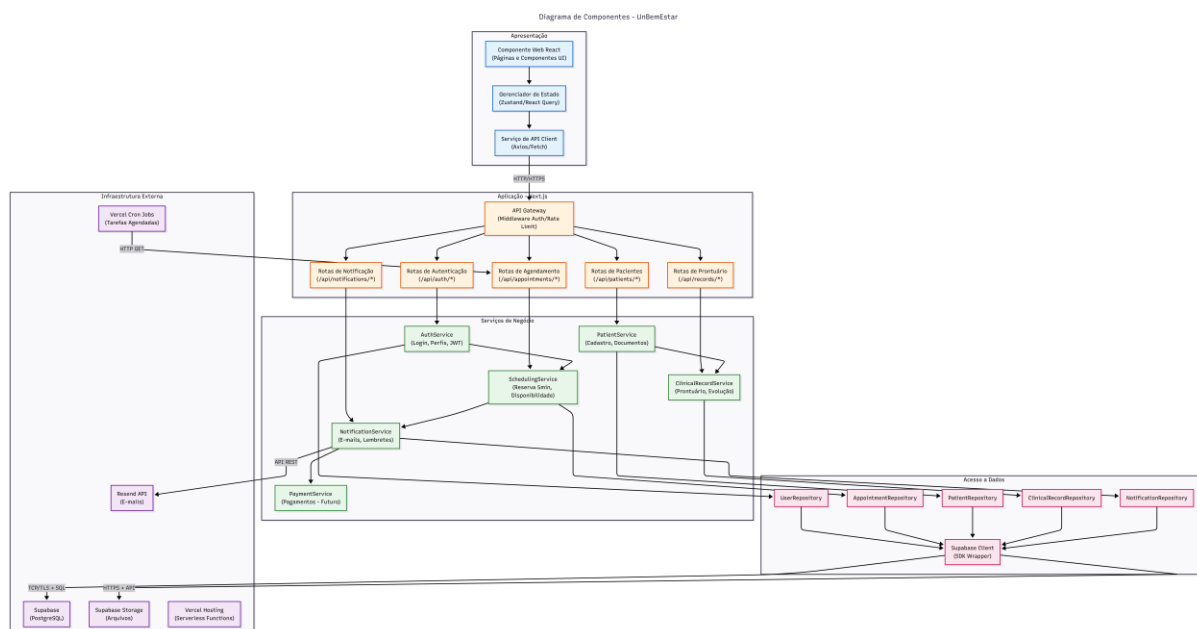
O diagrama apresenta as principais entidades do sistema (Usuário, Paciente, Secretaria, Fisioterapeuta, Administrador, Consulta, Prontuario, Evolucao, Documento, Agenda), seus atributos, métodos e relacionamentos. A classe abstrata `Usuario` é especializada nos quatro perfis de acesso.



Fonte: Elaborado por Brenda Maria Cavalcante Chaves (2026).

Figura 7 - Diagrama de Componentes

O diagrama organiza os componentes do sistema nas camadas de Apresentação (React), Aplicação (API Routes Next.js), Serviços de Negócio, Acesso a Dados (Repositories + Supabase Client) e Infraestrutura Externa (Supabase, Resend, Vercel). As setas indicam as dependências e os protocolos de comunicação.



Fonte: Elaborado por Brenda Maria Cavalcante Chaves (2026).

2.5 Visão de Implantação

O software UnBemEstar será implantado em uma infraestrutura totalmente baseada em nuvem, eliminando a necessidade de servidores físicos dedicados ou gerenciamento manual de infraestrutura. Essa escolha se justifica pela natureza do projeto: equipe acadêmica reduzida, prazo semestral e necessidade de entregas incrementais, além de estar alinhada com as decisões tecnológicas definidas no Documento de Visão.

Infraestrutura de hardware (nós de implantação): A aplicação não depende de servidores físicos próprios. Os nós de processamento são providos pelos serviços gerenciados descritos a seguir. O dispositivo do usuário final (PC ou smartphone) atua como cliente leve, executando apenas o navegador web e recebendo as páginas renderizadas pelo servidor.

Tecnologias de implantação:

- A. **Vercel (PaaS — Plataforma de Hospedagem e Deploy):** A aplicação Next.js é hospedada na Vercel, plataforma especializada em frameworks baseados em React. A Vercel provê provisionamento automático de infraestrutura serverless, distribuição global via CDN (Content Delivery Network) e Edge Middleware, garantindo baixa latência para usuários no Brasil. A justificativa para sua escolha é direta: a integração nativa com GitHub Actions permite pipeline de CI/CD (Integração e Entrega Contínua) totalmente automatizado, de modo que a cada push na branch principal um novo deploy é acionado automaticamente, o que está alinhado com a prática de Integração Contínua do XP adotada pela equipe (Seção 3.2 do Documento de Visão). Além disso, a Vercel Cron executa os jobs agendados responsáveis pelo disparo dos lembretes automáticos de consulta via e-mail (RF07).
- B. **Next.js com TypeScript (Framework Full-Stack):** O front-end e as rotas de back-end são implementados em um único projeto Next.js (TypeScript), conforme definido no Documento de Visão (Seção 1.6). O Next.js suporta Server-Side Rendering (SSR)

e API Routes, permitindo que o mesmo servidor responda tanto pelas páginas HTML quanto pelas chamadas de API REST, simplificando a arquitetura de deploy. A validação de dados de entrada é realizada pela biblioteca Zod, garantindo type safety em toda a camada de comunicação cliente e servidor.

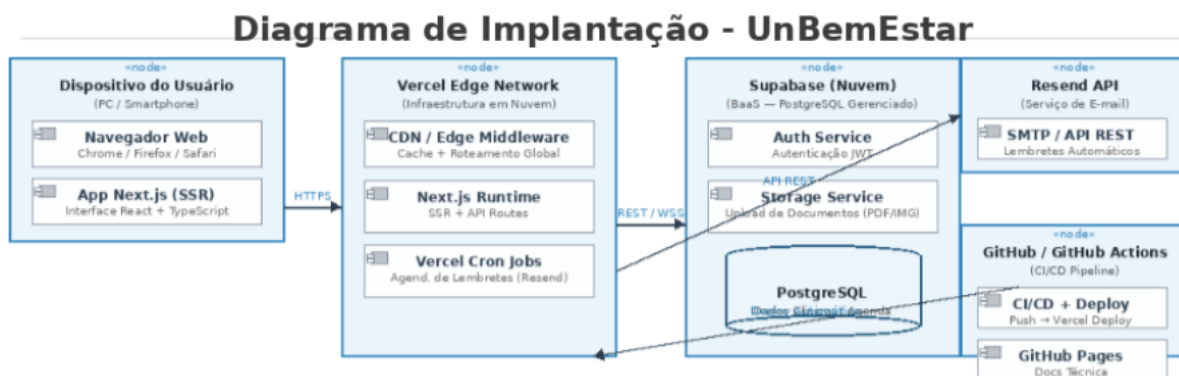
- C. **Resend (Serviço de E-mail Transacional):** O envio de e-mails transacionais (confirmações de agendamento e lembretes automáticos) é delegado ao serviço Resend, consumido via API REST pelos Cron Jobs da Vercel. Essa separação de responsabilidades garante alta entregabilidade dos e-mails sem sobrecarregar a aplicação principal.

Banco de dados: O sistema utiliza o **Supabase** como plataforma BaaS (Backend as a Service), que provê um banco de dados **PostgreSQL gerenciado**, serviço de autenticação (Auth Service com JWT) e armazenamento de arquivos (Storage Service). A escolha do Supabase se justifica por três razões principais: (i) elimina a necessidade de provisionamento e manutenção de servidor de banco de dados, reduzindo a carga operacional da equipe; (ii) o PostgreSQL é um sistema de gerenciamento de banco de dados relacional robusto, maduro e adequado ao domínio clínico do projeto, com suporte nativo a transações ACID essenciais para garantir integridade dos agendamentos; e (iii) a integração nativa com Next.js via SDK JavaScript simplifica o desenvolvimento das API Routes. O banco armazena os dados de pacientes, profissionais, agendamentos, horários e relatórios de evolução, em conformidade com os princípios da LGPD (Lei nº 13.709/2018), conforme requisito não funcional RNF01 do projeto.

Pipeline de CI/CD: O versionamento e o pipeline de entrega contínua são gerenciados pelo **GitHub** com **GitHub Actions**. A cada pull request aprovado e mergeado no branch principal, os testes automatizados (Vitest + Supertest) são executados e, em caso de sucesso, o deploy é disparado automaticamente para a Vercel (RNF05). A documentação técnica é publicada via GitHub Pages. Essa estratégia é consistente com a prática de Integração Contínua adotada no processo ScrumXP da equipe.

Figura 8 – Diagrama de Implantação

O diagrama mostra a distribuição física do sistema: o cliente (navegador) acessa a aplicação hospedada na Vercel (serverless functions + CDN), que se integra ao Supabase (banco de dados PostgreSQL, storage e autenticação), ao Resend (envio de e-mails) e aos Vercel Cron Jobs (tarefas agendadas). O pipeline de CI/CD via GitHub Actions automatiza os deploys.



Fonte: Elaborado por João Vitor Tavares (2026)

2.6 Restrições adicionais

Além das metas e restrições tecnológicas elencadas na seção 2.4, o sistema UnBemEstar deverá observar as seguintes restrições adicionais, derivadas de aspectos negociais, de qualidade e de segurança.

2.6.1 Restrições Negociais e de Contexto de Uso

Acesso via Internet com Autenticação Obrigatória: O software será acessível diretamente pela Internet (Web, SaaS). No entanto, todas as funcionalidades, exceto a página inicial institucional e a tela de recuperação de senha, exigirão autenticação prévia do usuário via e-mail e senha. Esta restrição é fundamental para cumprir a LGPD e garantir que apenas perfis autorizados (paciente, secretária, fisioterapeuta, admin) acessem dados clínicos sensíveis.

Escalabilidade Preliminar (Cenário Alvo): O sistema será projetado e testado para suportar, em sua primeira versão, até 5 clínicas de fisioterapia de pequeno porte, totalizando uma base de dados de aproximadamente 10.000 pacientes ativos e 50 profissionais, com uma carga de até 100 agendamentos por hora nos horários de pico (manhã e início da noite). Esta restrição orienta o dimensionamento das funções serverless na Vercel e os índices no Supabase, evitando sub ou superdimensionamento.

2.6.2 Restrições de Qualidade de Software

Usabilidade (Acessibilidade e Simplicidade): Conforme requisito RNF02, a interface deve atender usuários de 18 a 80+ anos com diferentes níveis de alfabetização digital. Como restrição adicional, o sistema deverá atingir nível AA das Diretrizes de Acessibilidade para Conteúdo Web (WCAG 2.1) no que tange a contraste de cores, navegação por teclado e compatibilidade com leitores de tela. A justificativa é garantir a inclusão de pacientes idosos ou com deficiência visual parcial, público comum em clínicas de fisioterapia.

Confiabilidade (Absenteísmo): A meta não é apenas técnica, mas de negócio. O sistema deve reduzir a taxa de faltas (no-shows) em pelo menos 30% em relação ao processo manual via WhatsApp/telefone, conforme expectativa do cliente no Documento de Visão. Para isso, a restrição técnica é que o serviço de lembrete automático (Vercel Cron + Resend)

deve ter 99,9% de entregabilidade de e-mails válidos, com retentativas em caso de falha temporária.

Portabilidade: O sistema deve ser executável e responsivo nos navegadores Firefox (versão estável mais recente), Chrome (versão estável mais recente), Edge (versão estável mais recente) e Safari (versão estável mais recente), em suas versões de desktop e mobile (iOS e Android). A restrição de portabilidade evita o lock-in em um único ecossistema (ex: Chrome) e respeita a diversidade de dispositivos dos usuários finais.

2.6.3 Restrições de Segurança e Permissões de Acesso

A segurança é uma meta crítica (RNF01). A Tabela 1 abaixo justifica e detalha as permissões de acesso, implementadas via Row Level Security (RLS) no Supabase e middleware na aplicação Next.js.

Tabela 1 - Perfis de Acesso e Permissões do Sistema UnBemEstar

Perfil	Responsabilidades e Justificativa	Permissões de Acesso (Restrições)
Paciente	Acessa apenas seus próprios dados. Justifica-se pela LGPD (autodeterminação informativa).	CRUD apenas sobre seus próprios agendamentos, documentos e perfil. Leitura apenas dos profissionais (nome, especialidade). Bloqueado acesso a prontuários de outros pacientes.
Secretária	Gerencia a agenda e o fluxo de pacientes. Justifica-se pela necessidade operacional da clínica.	CRUD sobre todos os agendamentos. Leitura de todos os pacientes e profissionais. Bloqueada a criação/edição de prontuários clínicos e a exclusão permanente de dados.
Fisioterapeuta/Médico	Acessa e registra dados clínicos (evolução). Justifica-se pela necessidade de assistência e continuidade do tratamento.	CRUD sobre prontuários e evoluções apenas dos seus pacientes. Leitura da agenda de seus horários. Bloqueada a

		alteração de dados financeiros ou de outro profissional.
Administrador	Mantém o sistema (cadastros, auditoria). Justifica-se pela necessidade de governança e conformidade.	CRUD global sobre usuários, profissionais, configurações e auditoria. Acesso total aos logs (quem acessou o quê). Bloqueada a violação de hashes de senha (armazenadas com bcrypt).

Fonte: Elaborado por Brenda Maria C. Chaves (2026)

3 Bibliografia

ALMEIDA, M. B.; ANDRADE, R. M. Engenharia de Software: uma abordagem prática. 4. ed. Rio de Janeiro: Elsevier, 2021.

BASS, L.; CLEMENTS, P.; KAZMAN, R. Software Architecture in Practice. 3. ed. Boston: Addison-Wesley, 2013.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 13 maio 2026.

FIELDING, R. T. Architectural Styles and the Design of Network-based Software Architectures. 2000. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Padrões de Projeto: Soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

MARTIN, R. C. Arquitetura Limpa: O guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SUPABASE. Supabase Documentation. 2026. Disponível em: <https://supabase.com/docs>. Acesso em: 13 maio 2026.

VERCEL. Vercel Documentation: Cron Jobs and Serverless Functions. 2026. Disponível em: <https://vercel.com/docs/cron-jobs>. Acesso em: 13 maio 2026.